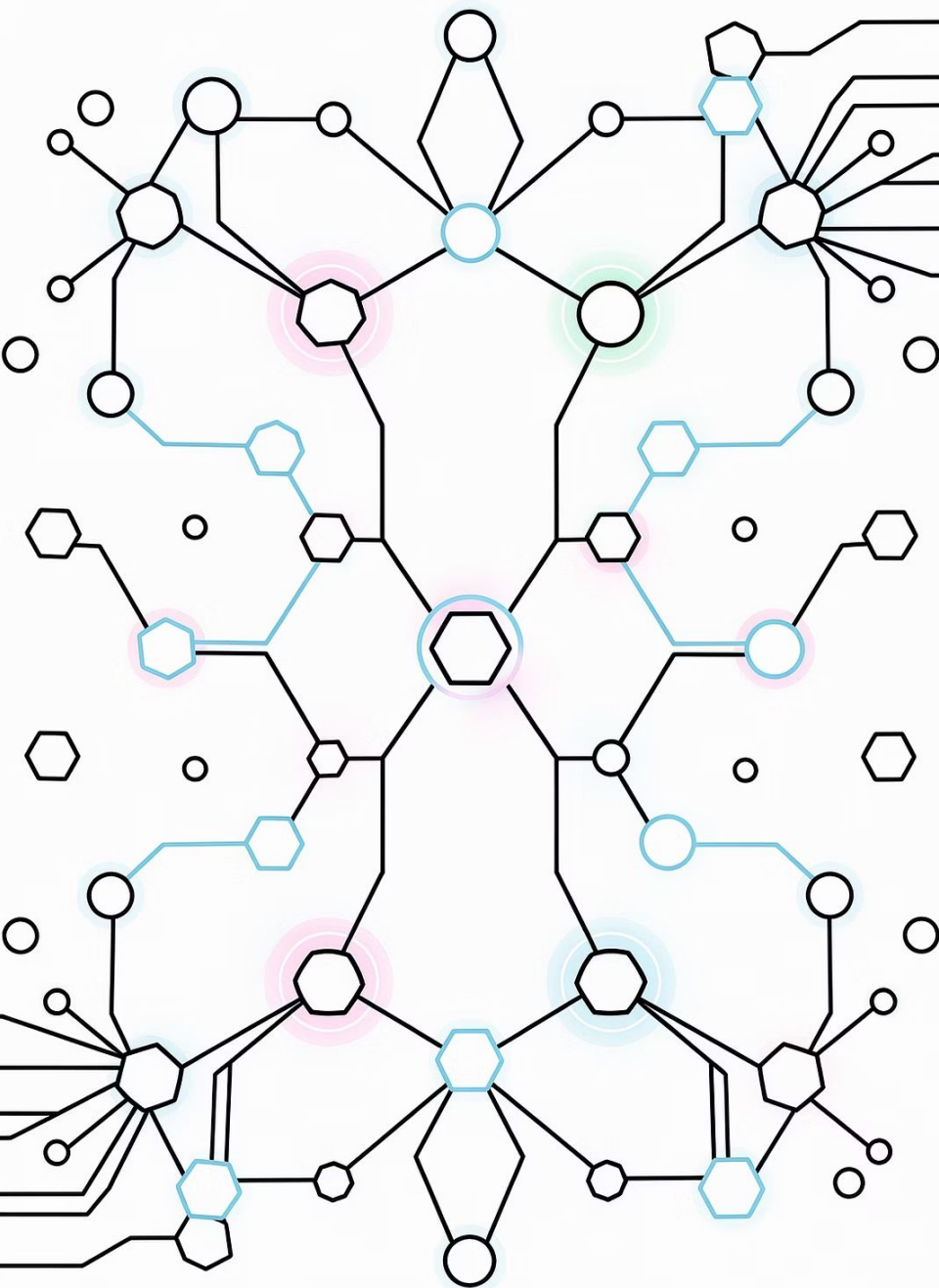




Deep Learning: Visão Geral

Deep Learning representa o aprendizado com redes neurais profundas, onde a característica central é o aprendizado automático de representações. Diferentemente do Machine Learning clássico, requer menos feature engineering e permite mais aprendizado fim-a-fim.

O marco histórico da consolidação prática ocorreu a partir de 2012-2013, impulsionado pela disponibilidade de grandes volumes de dados e pelo poder computacional das GPUs.

Eduardo Ogasawara

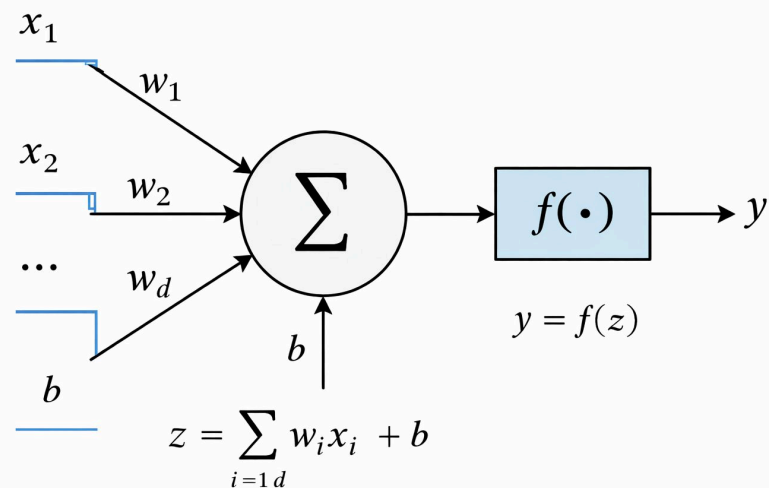
eduardo.ogasawara@cefet-rj.br
<https://eic.cefet-rj.br/~eogasawara>

Neurônio Artificial: O Modelo Básico

Um neurônio artificial computa uma soma ponderada das entradas e aplica uma função de não linearidade para produzir a saída.

$$z = \sum_{i=1}^d w_i x_i + b$$
$$y = f(z)$$

Onde **w***i* são os pesos, **b** é o viés, e **f**(•) é a função de ativação.



Perceptron e Suas Limitações

O Perceptron

Modelo de classificador linear que processa entradas através de uma única camada de neurônios.

Limitação Fundamental

Não consegue resolver problemas não linearmente separáveis, como o clássico problema XOR.

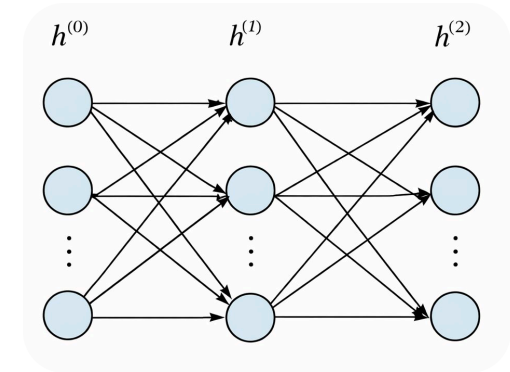
Motivação para Evolução

Essa limitação motivou o desenvolvimento de redes com múltiplas camadas, capazes de aprender representações mais complexas.

Redes Neurais Multicamadas (MLP)

As redes multicamadas empilham várias camadas de neurônios, onde cada camada aprende uma representação intermediária dos dados. Esta arquitetura permite capturar padrões complexos e não lineares.

$$\mathbf{h}^{(l)} = f \left(W^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)} \right)$$



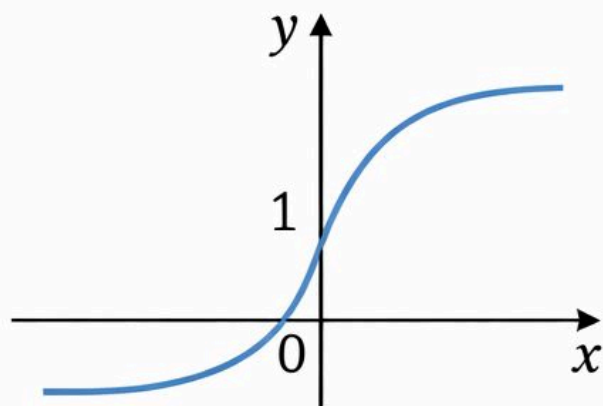
Cada camada transforma a representação da camada anterior, construindo abstrações progressivamente mais sofisticadas.

- ❏ Redes profundas aprendem representações hierárquicas: camadas iniciais capturam bordas e texturas, camadas intermediárias combinam padrões complexos, e camadas finais reconhecem objetos ou significado semântico — quanto mais profunda, maior o poder representacional.

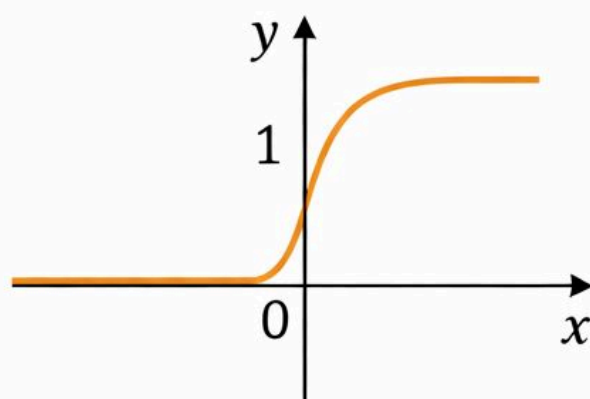
Funções de Ativação: A Essência da Não Linearidade

As funções de ativação introduzem não linearidade nas redes neurais, permitindo que aprendam padrões complexos. As principais funções incluem:

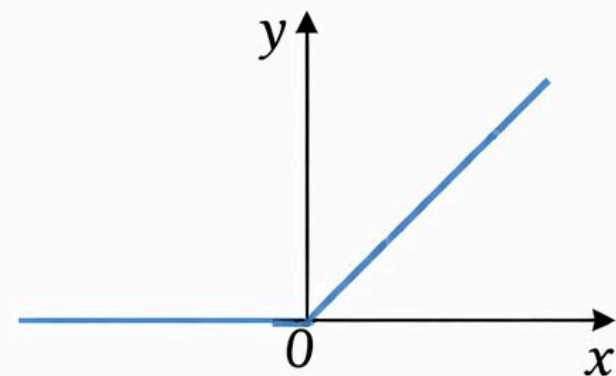
- **Sigmoid:** Comprime valores entre 0 e 1
- **Tanh:** Comprime valores entre -1 e 1
- **ReLU:** Padrão moderno, mitiga vanishing gradient $\text{ReLU}(x) = \max(0, x)$



$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



$$\text{ReLU}(x) = \max(0, x)$$

Função de Perda: O Objetivo do Aprendizado

Regressão: MSE

Mean Squared Error é utilizado para problemas de regressão, medindo o erro quadrático médio.

$$\mathcal{L}_{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Classificação: Cross-Entropy

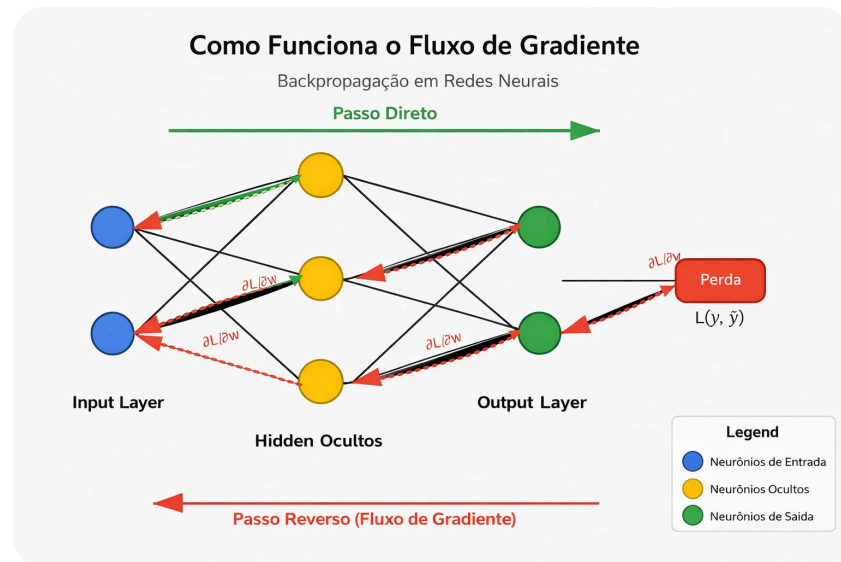
Cross-Entropy é a escolha padrão para problemas de classificação.

$$\mathcal{L}_{CE} = - \sum_k y_k \log(\hat{y}_k)$$

Aprendizado por Gradiente & Backpropagation

O objetivo é minimizar a função de perda $\mathcal{L}(\theta)$ via Gradient Descent. O Backpropagation calcula os gradientes eficientemente usando a regra da cadeia, propagando o erro da saída até as camadas iniciais.

$$\min_{\theta} \mathcal{L}(\theta)$$
$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$



Algoritmo: Treinamento por Backpropagation

Referência: Rumelhart et al. (1986)

ENTRADA: dados (X, y) , taxa η , épocas T

PRÉ-CONDIÇÃO: rede inicializada com pesos aleatórios

PARA época = 1 até T :

PARA cada mini-batch (x_b, y_b) :

Forward Pass

$$h^{(0)} \leftarrow x_b$$

PARA $l = 1$ até L :

$$z^{(l)} \leftarrow W^{(l)} h^{(l-1)} + b^{(l)}$$

$$h^{(l)} \leftarrow f(z^{(l)})$$

$$\hat{y} \leftarrow h^{(L)}$$

Cálculo da Perda

$$L \leftarrow \text{perda}(\hat{y}, y_b)$$

Backward Pass (regra da cadeia)

$$\delta^{(L)} \leftarrow \partial L / \partial z^{(L)}$$

PARA $l = L-1$ até 1 :

$$\delta^{(l)} \leftarrow (W^{(l+1)^T} \delta^{(l+1)}) \odot f'(z^{(l)})$$

Atualização dos Pesos

PARA $l = 1$ até L :

$$W^{(l)} \leftarrow W^{(l)} - \eta \cdot \delta^{(l)} (h^{(l-1)})^T$$

$$b^{(l)} \leftarrow b^{(l)} - \eta \cdot \delta^{(l)}$$

SAÍDA: pesos $W^{(l)}$, $b^{(l)}$ otimizados

PÓS-CONDIÇÃO: $\mathcal{L}(\theta)$ minimizada sobre os dados de treino

Redes Convolucionais: Motivação e Operação

Redes Neurais Convolucionais (CNNs) foram desenvolvidas para processar dados com estrutura espacial, como imagens, sinais e até texto.

Problema do MLP

Redes totalmente conectadas requerem um número excessivo de parâmetros para processar imagens, tornando o treinamento impraticável.

Solução da CNN

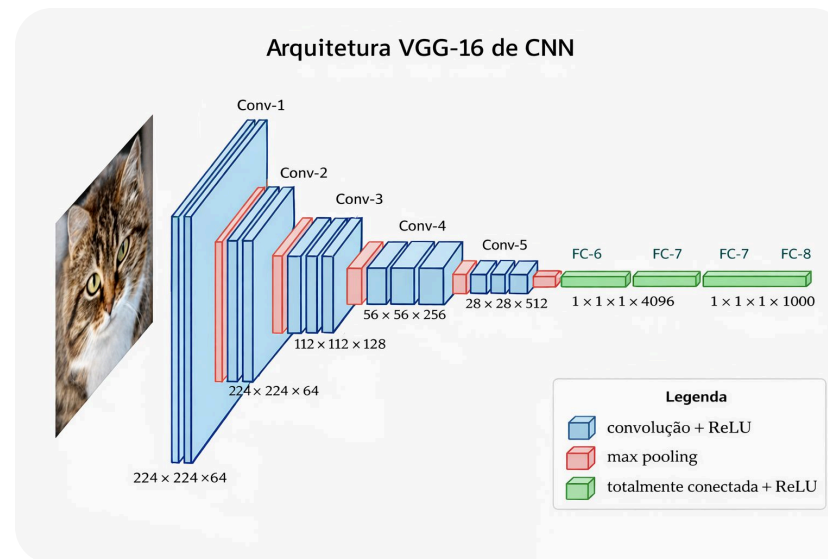
CNNs exploram a localidade espacial e o compartilhamento de pesos, reduzindo drasticamente o número de parâmetros.

Convolução: A Ideia Matemática

A operação de convolução aplica um kernel (filtro) localmente sobre a entrada:

$$(\mathbf{x} * \mathbf{k})(i) = \sum_j \mathbf{x}_{i+j} \mathbf{k}_j$$

A mesma operação é aplicada em toda a entrada através do compartilhamento de pesos, o que reduz drasticamente o número de parâmetros comparado a camadas totalmente conectadas.



Pooling e Invariância



Agregação Local

Pooling realiza agregação local, como max-pooling, que seleciona o valor máximo em cada região.



Redução Dimensional

Reduz a dimensionalidade espacial dos dados, diminuindo o custo computacional.



Invariância

Introduz invariância a pequenas translações, tornando a rede mais robusta.

RNN e LSTM: Memória em Sequências

RNNs processam sequências mantendo um estado oculto h_t que carrega memória do passado. Porém sofrem com gradiente desvanecente em sequências longas.

$$h_t = f(W_x x_t + W_h h_{t-1})$$

LSTM resolve isso com três portas de controle do fluxo de informação.

Porta de Esquecer

Descarta memória irrelevante

Porta de Atualizar

Armazena nova informação

Porta de Saída

Expõe o estado h_t

Algoritmo: LSTM Forward Pass

Referência: Hochreiter & Schmidhuber (1997)

ENTRADA: sequência $x_1, x_2, \dots, x_T$

PRÉ-CONDIÇÃO: $h_0 = 0, c_0 = 0$

PARA $t = 1$ até T :

Porta de Esquecer

$f_t \leftarrow \text{sigma}(W_f \cdot [h_{t-1}, x_t] + b_f)$

Porta de Atualizar

$i_t \leftarrow \text{sigma}(W_i \cdot [h_{t-1}, x_t] + b_i)$

$c_{\sim t} \leftarrow \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$

Estado da Célula

$c_t \leftarrow f_t * c_{t-1} + i_t * c_{\sim t}$

Porta de Saída

$o_t \leftarrow \text{sigma}(W_o \cdot [h_{t-1}, x_t] + b_o)$

$h_t \leftarrow o_t * \tanh(c_t)$

SAÍDA: $h_1, h_2, \dots, h_T$

PÓS-CONDIÇÃO: gradientes fluem via c_t sem desvanecimento

Por Que Grafos São Diferentes

Dados Não-Euclidianos

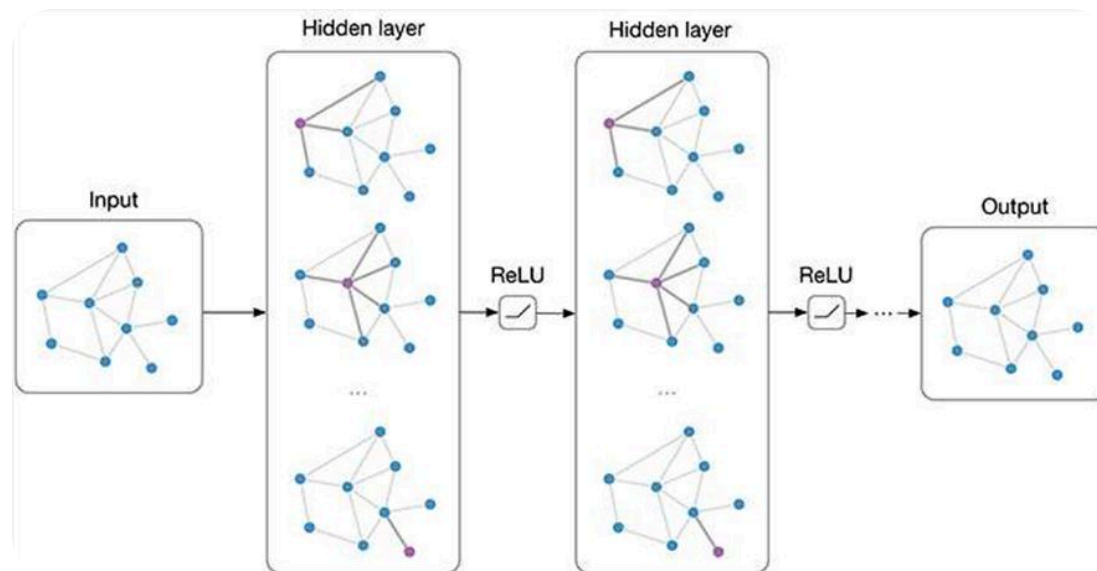
Grafos não possuem estrutura de grade regular como imagens ou sequências.

Vizinhanças Irregulares

Cada nó pode ter um número diferente de vizinhos, sem padrão fixo.

Invariância à Permutação

A ordem dos nós não importa - o grafo permanece o mesmo sob permutações.



GNNs: Message Passing & GraphSAGE

A ideia central das GNNs é o paradigma de **message passing** — cada nó agrega informação de seus vizinhos iterativamente.

$$\mathbf{h}_v^{(k+1)} = \text{UPDATE} \left(\mathbf{h}_v^{(k)}, \text{AGG} \{ \mathbf{h}_u^{(k)} : u \in \mathcal{N}(v) \} \right)$$

GraphSAGE torna isso escalável com amostragem de vizinhos e funções de agregação explícitas (média, max-pooling)

Algoritmo: GraphSAGE Forward Pass
Referência: Hamilton et al. (2017).

ENTRADA: grafo $G=(V,E)$, features $\mathbf{h}_v^0 = \mathbf{x}_v$, camadas K

PRÉ-CONDIÇÃO: vizinhança $\mathcal{N}(v)$ definida para todo v

PARA $k = 1$ até K :

PARA cada nó v em V :

Amostragem de Vizinhos

$\mathcal{N}_s(v) \leftarrow \text{SAMPLE}(\mathcal{N}(v), \text{tamanho } S)$

Agregação

$\mathbf{h}_{\mathcal{N}(v)}^k \leftarrow \text{AGG}_k \{ \mathbf{h}_u^{k-1} : u \in \mathcal{N}_s(v) \}$

AGG pode ser: média, max-pooling ou LSTM

Atualização (concatenação + transformação)

$\mathbf{h}_v^k \leftarrow \text{ReLU}(W^k \cdot \text{CONCAT}(\mathbf{h}_v^{k-1}, \mathbf{h}_{\mathcal{N}(v)}^k))$

Normalização L2

PARA cada v : $\mathbf{h}_v^k \leftarrow \mathbf{h}_v^k / \|\mathbf{h}_v^k\|_2$

SAÍDA: embeddings $\mathbf{h}_v^K$ para cada nó v

PÓS-CONDIÇÃO: cada nó codifica info de vizinhos até K saltos

Atenção e Graph Attention Networks (GAT)

Mecanismo de Atenção Geral

O mecanismo de atenção reconhece que **nem toda informação passada é igualmente relevante**. Em vez de comprimir toda a sequência em um único vetor, ele pondera dinamicamente as contribuições de diferentes partes da entrada:

$$c_j = \sum_i \alpha_{ji} h_i$$

Os pesos **α_{ji}** são **aprendidos pela rede**, indicando quais partes da entrada merecem mais atenção para cada saída gerada.

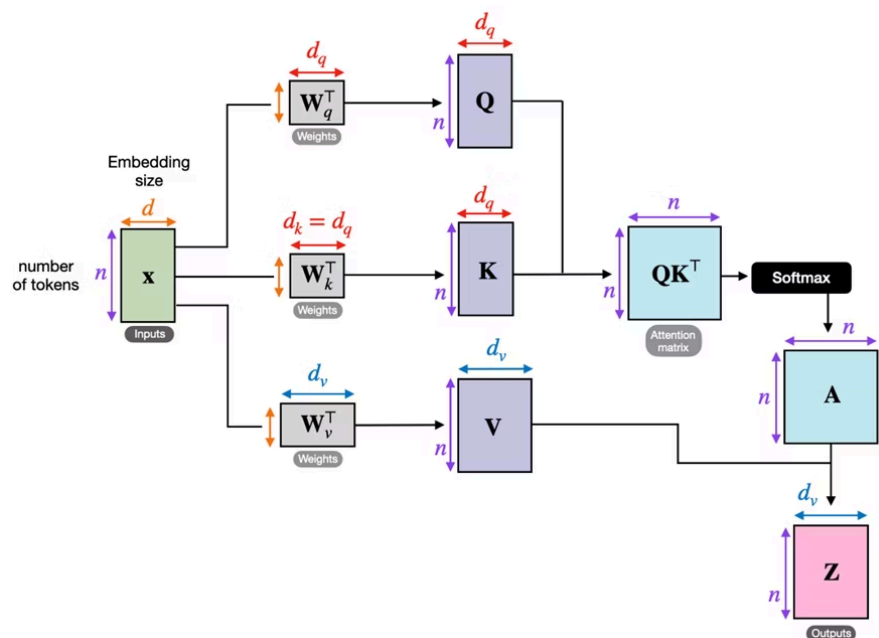
Graph Attention Networks (GAT)

GAT generaliza o conceito de atenção para grafos, permitindo que cada nó aprenda a **importância relativa de seus vizinhos**:

$$\alpha_{vu} = \text{softmax}(a(\mathbf{h}_v, \mathbf{h}_u))$$

Os pesos de atenção **α_{vu}** são aprendidos dinamicamente, indicando quais vizinhos são mais relevantes para cada nó. Isso permite que a rede foque nas conexões mais importantes do grafo.

- ❑ GAT combina o poder da atenção com a estrutura de grafos, oferecendo flexibilidade e interpretabilidade superior em comparação com métodos de agregação fixa.



Referências Principais

Esta seleção de referências representa os pilares fundamentais para o estudo aprofundado de mineração de dados, cobrindo desde conceitos básicos até técnicas avançadas e aplicações contemporâneas.



1. **J. Han, J. Pei, and H. Tong**, *Data Mining: Concepts and Techniques*, 4th edition. Cambridge, MA: Morgan Kaufmann, 2022.
2. **G. M. James, D. Witten, T. Hastie, and R. Tibshirani**, *An Introduction to Statistical Learning: With Applications in R*. Springer Nature, 2021.
3. **S. Garcia, J. Luengo, and F. Herrera**, *Data Preprocessing in Data Mining*. Springer, 2014.
4. **C. M. Bishop and H. Bishop**, *Deep Learning: Foundations and Concepts*. Springer Nature, 2023.